DSA INTERVIEW COMPLETE SYLLABUS

Beginner → Intermediate → Advanced → Interview Ready

Preparation reference for software engineering interviews at Microsoft, Google, Amazon and Qualcomm — and other product-based companies.

How to use this booklet
1. Learn the concept before memorising solutions.
2. Implement each data structure once without copying.
3. Solve problems from easy to medium to hard.
4. Revisit mistakes and explain your approach aloud.
5. Tick a box only when you can solve and explain the topic independently.

A note about company interviews

Interview questions vary by role, team, location and hiring cycle. No public list can guarantee every question. This booklet is a broad preparation syllabus, not an official or exhaustive company question bank.

CORE ROADMAP

Phase 1: Programming foundations and complexity
Phase 2: Linear data structures, hashing, sorting and searching
Phase 3: Trees, heaps and graphs
Phase 4: Recursion, backtracking, greedy and dynamic programming
Phase 5: Advanced structures, problem-solving patterns and mock interviews

Name: _____ Start date: _____

1. Foundations, Arrays & Strings

A. Programming and problem-solving foundations

- ■ Choose one interview language: C++, Java or Python
- ■ Variables, data types, operators and expressions
- ■ Conditionals, loops and functions
- ■ Arrays, strings and basic input/output
- ■ Pointers/references and memory basics
- ■ Recursion and call stack
- ■ Classes, objects and basic OOP
- ■ Debugging, edge cases and test cases
- ■ Integer overflow and numeric limits
- ■ Pass-by-value vs pass-by-reference

B. Complexity analysis

- ■ Time complexity: Big-O, Big-Theta, Big-Omega
- ■ Space complexity and auxiliary space
- ■ Best, average and worst case
- ■ Constant, logarithmic, linear, n log n, quadratic
- ■ Amortized analysis
- ■ Recurrence relations (basic)
- ■ Trade-offs: time vs memory
- ■ Complexity of common operations

C. Arrays and matrices

- ■ Array operations and traversal
- ■ Insert, delete, update and search
- ■ Two pointers
- ■ Sliding window: fixed and variable size
- ■ Prefix sum and suffix sum
- ■ Difference array
- ■ Kadane's algorithm / maximum subarray
- ■ Sorting-based array problems
- ■ Merge two sorted arrays
- ■ Rotate and rearrange arrays
- ■ Next permutation
- ■ Missing/duplicate numbers
- ■ Subarray and subsequence problems
- ■ 2D arrays and matrix traversal
- ■ Spiral traversal and transpose
- ■ In-place matrix operations

D. Strings

- ■ String operations and immutability
- ■ Frequency counting and character maps
- ■ Palindrome and anagram checks
- ■ Substring vs subsequence
- ■ Longest substring without repetition
- ■ String matching: naive approach
- ■ KMP and Z algorithm
- ■ Rabin-Karp rolling hash
- ■ Manacher's algorithm (advanced)
- ■ String compression and parsing
- ■ Pattern matching and wildcard basics
- ■ Trie-based prefix search

2. Linear Data Structures, Hashing & Search

A. Linked lists

- ■ Singly linked list
- ■ Doubly linked list
- ■ Circular linked list
- ■ Insertion and deletion
- ■ Reverse a linked list: iterative and recursive
- ■ Find middle node (slow/fast pointers)
- ■ Cycle detection and cycle start (Floyd)
- ■ Merge two sorted linked lists
- ■ Remove nth node from end
- ■ Intersection of two linked lists
- ■ Add numbers represented by lists
- ■ LRU cache design

B. Stacks, queues and deques

- ■ Stack operations and applications
- ■ Queue and circular queue
- ■ Deque (double-ended queue)
- ■ Implement stack using queues
- ■ Implement queue using stacks
- ■ Balanced parentheses / valid brackets
- ■ Infix, postfix and prefix expressions
- ■ Next greater/smaller element
- ■ Monotonic stack
- ■ Monotonic queue
- ■ Largest rectangle in histogram
- ■ Sliding-window maximum

C. Hashing

- ■ Hash functions and hash tables
- ■ Collision handling: chaining and probing
- ■ HashMap / HashSet operations
- ■ Frequency maps
- ■ Two Sum and pair-sum patterns
- ■ Group anagrams
- ■ Longest consecutive sequence
- ■ Subarray sum equals K
- ■ Prefix sum + hashmap
- ■ Custom keys and equality basics

D. Searching and sorting

- ■ Linear and binary search
- ■ Binary search on answer
- ■ First/last occurrence and lower/upper bound
- ■ Search in rotated sorted array
- ■ Peak element and bitonic arrays
- ■ Search in 2D matrix
- ■ Bubble, selection and insertion sort
- ■ Merge sort and quicksort
- ■ Heap sort and counting sort
- ■ Radix sort and bucket sort
- ■ Stable vs unstable sorting
- ■ Comparator / custom sorting

3. Trees, Heaps & Tries

A. Binary trees

- Tree terminology, height, depth and levels
- Preorder, inorder and postorder traversal
- Recursive and iterative traversals
- Level-order / BFS traversal
- Zigzag and boundary traversal
- Height, diameter and balanced-tree check
- Maximum path sum
- Lowest common ancestor
- Root-to-leaf and path-sum problems
- Views: left, right, top and bottom
- Serialize and deserialize a tree
- Build tree from traversal orders
- Flatten binary tree to linked list

B. Binary search trees and balanced trees

- BST search, insert and delete
- Validate a BST
- Kth smallest/largest in BST
- Predecessor and successor
- LCA in BST
- Recover swapped BST nodes
- AVL tree concepts and rotations
- Red-Black tree concepts
- Balanced vs unbalanced trees

C. Heaps and priority queues

- Min heap and max heap
- Heapify and build heap
- Insert, extract and peek
- Heap sort
- Kth largest/smallest element
- Top K frequent elements
- Merge K sorted lists/arrays
- Find median from data stream
- Two heaps pattern
- Priority queue applications

D. Tries and prefix structures

- Trie node structure and operations
- Insert, search and prefix search
- Word dictionary / wildcard search
- Autocomplete and suggestions
- Longest common prefix
- Word search with trie
- Binary trie for XOR queries
- Compressed trie concepts

4. Graphs

A. Graph fundamentals and traversal

- ■ Vertices, edges, degree and paths
- ■ Directed vs undirected graphs
- ■ Weighted vs unweighted graphs
- ■ Adjacency list, matrix and edge list
- ■ BFS and DFS
- ■ Connected components
- ■ Cycle detection: directed and undirected
- ■ Bipartite graph checking
- ■ Topological sorting: DFS and Kahn's algorithm
- ■ Grid/2D graph traversal
- ■ Multi-source BFS

B. Shortest paths and spanning trees

- ■ Dijkstra's algorithm
- ■ Bellman-Ford algorithm
- ■ Floyd-Warshall algorithm
- ■ 0-1 BFS
- ■ Minimum spanning tree
- ■ Kruskal's algorithm
- ■ Prim's algorithm
- ■ Disjoint Set Union (Union-Find)
- ■ Path compression and union by rank/size

C. Advanced graph algorithms

- ■ Strongly connected components
- ■ Kosaraju's and Tarjan's algorithms
- ■ Bridges and articulation points
- ■ Euler path and circuit
- ■ Hamiltonian path (concepts / hard problems)
- ■ Network flow and max-flow basics
- ■ Bipartite matching basics
- ■ Shortest path in DAG
- ■ A* search (conceptual)
- ■ DSU on graphs / offline connectivity

Graph practice should include disconnected graphs, duplicate edges, self-loops, large input sizes and unreachable nodes.

5. Recursion, Backtracking, Greedy & Dynamic Programming

A. Recursion and divide & conquer

- Base case and recursive relation
- Recursion tree and call stack
- Tail recursion (concept)
- Subsets and subsequences
- Permutations and combinations
- Merge sort and quicksort
- Binary search recursion
- Inversion count
- Closest pair of points (advanced)
- Divide-and-conquer recurrence basics

B. Backtracking

- Choose-explore-unchoose pattern
- N-Queens
- Sudoku solver
- Rat in a maze
- Generate subsets and permutations
- Combination sum
- Palindrome partitioning
- Word search
- Graph coloring
- Pruning and constraint ordering

C. Greedy algorithms

- Greedy choice and exchange argument
- Activity / interval selection
- Merge intervals
- Meeting rooms and scheduling
- Fractional knapsack
- Minimum platforms / resource allocation
- Huffman coding
- Job sequencing
- Jump game
- Gas station
- Minimum coins (when greedy is valid)
- Minimum spanning tree algorithms

D. Dynamic programming

- Memoization vs tabulation
- State, transition and base cases
- 1D DP and Fibonacci-style problems
- 0/1 and unbounded knapsack
- Coin change: min coins and number of ways
- Subset sum and partition equal subset
- Longest common subsequence
- Longest increasing subsequence
- Edit distance
- Longest palindromic subsequence
- DP on grids and paths
- DP on trees
- Interval DP / matrix chain multiplication
- Bitmask DP
- Digit DP (advanced)
- Space optimization

6. Bit Manipulation, Mathematics & Advanced Structures

A. Bit manipulation

- ■ Binary representation and bitwise operators
- ■ Set, clear, toggle and check a bit
- ■ Count set bits / Brian Kernighan
- ■ Power of two
- ■ XOR properties and single number
- ■ Bit masks and subset generation
- ■ Bit shifts and signed integers
- ■ XOR prefix / range queries
- ■ Bitwise trie
- ■ Bitmask DP

B. Mathematics for DSA

- ■ GCD and LCM: Euclidean algorithm
- ■ Prime checking and sieve of Eratosthenes
- ■ Segmented sieve (advanced)
- ■ Prime factorization
- ■ Modular arithmetic
- ■ Fast exponentiation
- ■ Modular inverse (advanced)
- ■ Factorials and combinations
- ■ Pascal's triangle
- ■ Permutations and combinations
- ■ Pigeonhole principle (basic)
- ■ Probability and expected value basics

C. Advanced data structures

- ■ Disjoint Set Union (DSU)
- ■ Fenwick Tree / Binary Indexed Tree
- ■ Segment Tree: build, query, update
- ■ Lazy propagation
- ■ Sparse Table and range queries
- ■ Monotonic data structures
- ■ Ordered sets / balanced search trees
- ■ Order statistic tree (concept)
- ■ Suffix array and LCP
- ■ Suffix automaton (advanced)
- ■ Persistent data structures (concept)
- ■ Treap / B-tree concepts

D. Other useful patterns

- ■ Fast and slow pointers
- ■ Merge intervals
- ■ Sweep line
- ■ Coordinate compression
- ■ Meet in the middle
- ■ Offline queries
- ■ Randomized algorithms basics
- ■ Reservoir sampling (advanced)
- ■ Line sweep and geometry basics
- ■ Prefix/suffix techniques

7. Interview Readiness & Company Focus

A. Problem-solving patterns to recognise

- Two pointers
- Sliding window
- Prefix sum + hashmap
- Fast and slow pointers
- Binary search on answer
- Monotonic stack / queue
- Top K with heap
- BFS / DFS
- Topological sort
- Union-Find
- Greedy intervals
- Backtracking
- 1D / 2D / tree DP
- Trie and bitwise XOR
- Sweep line / difference array

B. Interview execution checklist

- Clarify input, output and constraints
- Ask about edge cases
- Explain brute force first
- Improve to an efficient approach
- State time and space complexity
- Write clean, readable code
- Dry-run with a sample
- Test empty, minimum and maximum cases
- Handle overflow and invalid assumptions
- Discuss trade-offs and alternatives

C. Common interview stages (not universal)

Stage	Typical focus
Online assessment	Timed coding, arrays, strings, hashing, sorting, searching, basic DP and aptitude where applicable
Technical coding rounds	DSA problem solving, data structures, algorithms, code quality and complexity
Design round(s)	Low-level design and/or system design depending on role and experience
Behavioral / hiring manager	Projects, teamwork, communication, ownership, learning and decision-making

D. Company-wise preparation emphasis

Company	Useful areas to prepare (role-dependent)
Microsoft	DSA fundamentals, trees/graphs, problem explanation, clean code; design and behavioral discussion may appear.
Google	Strong algorithmic reasoning, graphs, DP, complexity, edge cases and explaining trade-offs.
Amazon	DSA plus Leadership Principles / behavioral examples; arrays, trees, graphs, heaps and practical problem solving.
Qualcomm	DSA plus role-specific C/C++, operating systems, computer architecture, concurrency and embedded/systems fundamentals where relevant.

8. Personal Study Tracker

Suggested order — move forward when you can explain the idea, implement it, analyse complexity and solve a few variations.

Level	Study focus	Ready when…
1. Beginner	Language basics, complexity, arrays, strings, sorting, searching	You can solve straightforward problems without hints.
2. Core	Hashing, linked lists, stacks, queues, two pointers, sliding window	You can identify common patterns and handle edge cases.
3. Intermediate	Trees, BST, heaps, recursion, backtracking, graphs	You can choose the right traversal/data structure.
4. Advanced	Greedy, DP, advanced graphs, tries, bit manipulation	You can derive states/transitions and justify correctness.
5. Interview ready	Timed sets, mixed problems, mocks, review and communication	You can solve unfamiliar problems and explain trade-offs.

Weekly review checklist

■ I revised previously learned patterns.

■ I solved problems without looking at the editorial first.

■ I wrote down mistakes and the reason behind each mistake.

■ I practised explaining my approach out loud.

■ I reviewed time and space complexity.

■ I attempted at least one timed mixed-topic set.

■ I revisited one difficult problem after a gap.

Problem-practice sources

Use a mix of LeetCode, GeeksforGeeks, Codeforces and company-tagged practice sets. Treat tags as hints, not guarantees. Prefer understanding a pattern over memorising a specific solution.

Final reminder

The goal is not to tick every box quickly. Build strong fundamentals, practise consistently, learn from mistakes and communicate your reasoning clearly. This checklist is intentionally broad; depth matters more than rushing through every advanced topic.